

GNOME Shell Extensions Review Guidelines

TIP

If this is your first time writing an extension, please see the [Getting Started](#) guide.

These are the guidelines for developers who would like their extensions distributed on the [extensions.gnome.org](#). Extensions are reviewed carefully for malicious code, malware and security risks, but not for bugs.

General Guidelines

There are three basic guidelines for the operation of an extension:

1. Don't create or modify anything before `enable()` is called
2. Use `enable()` to create objects, connect signals and add `main` loop sources
3. Use `disable()` to cleanup anything done in `enable()`

These general tips will help your extension pass review:

- Write clean code, with consistent indentation and style
- Use modern features like ES6 classes and `async / await`
- Use a linter to check for logic and syntax errors
- Ask other developers for advice on [Matrix](#)

Rules

Only use initialization for static resources

Extensions **MUST NOT** create any objects, connect any signals, add any **main** loop sources or modify GNOME Shell during initialization.

In GNOME 45 and later, initialization happens when `extension.js` is imported and the `Extension` class is constructed (e.g. `constructor()` is called).

▼ GNOME Shell 45 and later

```
import {Extension} from 'resource:///org/gnome/shell/extensions/extension.js';  
  
export default class ExampleExtension extends Extension {  
  constructor(metadata) {  
    super(metadata);  
  
    // DO NOT create objects, connect signals or add main loop sources here  
  }  
  
  enable() {  
    // Create objects, connect signals, create main loop sources, etc.  
  }  
  
  disable() {  
    // Destroy objects, disconnect signals, remove main loop sources, etc.  
  }  
}
```

In GNOME 44 and earlier, initialization happens when `extension.js` is imported and the `init()` function is called. When using the `Extension` class pattern, the `constructor()` is also called.

▼ GNOME Shell 44 and earlier

```
const ExtensionUtils = imports.misc.extensionUtils;  
  
class Extension {  
  constructor() {  
    // DO NOT create objects, connect signals or add main loop sources here  
  }  
}
```

```
enable() {  
    // Create objects, connect signals, create main loop sources, etc.  
}  
  
disable() {  
    // Destroy objects, disconnect signals, remove main loop sources, etc.  
}  
  
}  
  
function init() {  
    // Initialize translations before returning the extension object  
    ExtensionUtils.initTranslations();  
  
    return new Extension();  
}
```

Extensions **MAY** create static data structures and instances of built-in JavaScript objects (such as `RegExp()` and `Map()`), but all dynamically stored memory must be cleared or freed in `disable()` (e.g. `Map.prototype.clear()`). All GObject classes, such as `Gio.Settings` and `St.Widget` are disallowed.

► Static Objects

Destroy all objects

Any objects or widgets created by an extension **MUST** be destroyed in `disable()` .

► GNOME 45 and later

► GNOME 44 and earlier

Disconnect all signals

Any signal connections made by an extension **MUST** be disconnected in `disable()` :

► **GNOME 45 and later**

► **GNOME 44 and earlier**

Remove **main** loop sources

Any **main** loop sources created **MUST** be removed in `disable()` .

Note that all sources **MUST** be removed in `disable()` , even if the callback function will eventually return `false` or `GLib.SOURCE_REMOVE` .

► **GNOME 45 and later**

► **GNOME 44 and earlier**

Do not use deprecated modules

Extensions **MUST NOT** import deprecated modules.

Deprecated Module	Replacement
<code>ByteArray</code>	<code><u>TextDecoder</u></code> and <code><u>TextEncoder</u></code>
<code>Lang</code>	<code><u>ES6 Classes</u></code> and <code><u>Function.prototype.bind()</u></code>
<code>mainloop</code>	<code><u>GLib.timeout_add()</u></code> , <code><u>setTimeout()</u></code> , etc

Do not import GTK libraries in GNOME Shell

Extensions **MUST NOT** import `gdk` , `gtk` or `adw` in the GNOME Shell process.

These libraries are only for use in the preferences process (`prefs.js`) and will conflict with `clutter` and other libraries used by GNOME Shell.

Do not import GNOME Shell libraries in Preferences

Extensions **MUST NOT** import `clutter` , `Meta` , `St` or `Shell` in the preferences process.

These libraries are only for use in the GNOME Shell process (`extension.js`) and will conflict with `gtk` and other libraries.

Avoid interfering with the Extension System

Extensions which modify, reload or interact with other extensions or the extension system are generally discouraged.

While not strictly prohibited, these extensions will be reviewed on a case-by-case basis and may be rejected at the reviewer's discretion.

Code must not be obfuscated

Extension code **MUST** be readable and reviewable JavaScript.

A specific code-style is not enforced, however submitted code must be formatted in a way that can be easily reviewed. The following rules **MUST** be adhered to:

- JavaScript code must be readable and reasonably structured
- JavaScript code must not be minified or obfuscated
- TypeScript must be transpiled to well-formatted JavaScript

No excessive logging

Extension **MUST NOT** print excessively to the log. The log should only be used for important messages and errors.

If a reviewer determines that an extension is writing excessively to the log, the extension will be rejected.

Extensions should not force dispose a GObject

Extensions **SHOULD NOT** call `GObject.Object.run_dispose()` unless absolutely necessary.

If absolutely necessary, any call to this method **MUST** have a comment explaining the real-world situation that makes it a requirement.

Scripts and Binaries

Use of external scripts and binaries is strongly discouraged. In cases where this is unavoidable for the extension to serve its purpose, the following rules must be adhered to:

- Extensions **MUST NOT** include binary executables or libraries
- Processes **MUST** be spawned carefully and exit cleanly
- Scripts **MUST** be written in GJS, unless absolutely necessary
- Scripts must be distributed under an OSI approved license

Reviewing Python modules, HTML, and web JavaScript dependencies is out of scope for extensions.gnome.org. Unless required functionality is only available in another scripting language, scripts must be written in GJS.

Extensions may install modules from well-known services such as `pip`, `npm` or `yarn` but **MUST** require explicit user action. For example, the extension preferences may include a page which describes the modules to be installed with a button.

Clipboard Access must be declared

Extensions that access the clipboard, with or without user interaction, **MUST** declare it in the description.

An extension **MUST NOT** share clipboard data with a third-party without explicit user interaction (e.g. button click, a user-defined keyboard shortcut).

An extension **MUST NOT** ship with default keyboard shortcuts for interacting with clipboard data.

Privileged Subprocess must not be user-writable

Spawning privileged subprocesses should be avoided at all costs.

If absolutely necessary, the subprocess **MUST** be run with `pkexec` and **MUST NOT** be an executable or script that can be modified by a user process.

Extensions must be functional

Extensions are reviewed, but not always tested for functionality so an extension **MAY** be approved with broken functionality or inoperable preferences window.

However, if an extension is tested and found to be fundamentally broken it will be rejected. Extensions which serve no purpose or have no functionality will also be rejected.

Extensions must not be AI-generated

While it is **not** prohibited to use AI as a learning aid or a development tool (i.e. code completions), extension developers should be able to justify and explain the code they submit, within reason.

Submissions with large amounts of unnecessary code, inconsistent code style, imaginary API usage, comments serving as LLM prompts, or other indications of AI-generated output will be rejected.

metadata.json must be well-formed

The `metadata.json` file that ships with every extension should be well-formed and accurately reflect the extension **without using any unnecessary keys**.

Key	Description
name	This should not conflict with another extension if possible. If it is a fork of another extension it MUST have a unique name to distinguish it.
uuid	This must be of the form <code>extension-id@namespace</code> . <code>extension-id</code> and <code>namespace</code> MUST only contain numbers, letters, period (<code>.</code>), underscore (<code>_</code>) and dash (<code>-</code>). Extensions MUST NOT use <code>gnome.org</code> as the namespace, but may use a registered web domain or accounts such as <code>username.github.io</code> and <code>username.gmail.com</code> .
description	This should be a reasonable length, but may contain a few paragraphs separated with <code>\n</code> literals or a bullet point list made with <code>*</code> characters.
version	Deprecated: This field is set for internal use by <code>extensions.gnome.org</code> .

Key	Description
shell-version	This MUST only contain stable releases and up to one development release. Extensions must not claim to support future GNOME Shell versions. As of GNOME 40, an entry may simply be a major version like 40 to cover the entire release.
url	This should be a link to a Github or <u>GitLab</u> repository where users can report problems and learn more about your extension.
session-modes	This MUST be dropped if you are only using user mode. The only valid values are user and unlock-dialog .
donations	This MUST only contain possible keys and MUST be dropped if you don't use any of the keys.

Example:

json

```
{
  "uuid": "color-button@my-account.github.io",
  "name": "ColorButton",
  "description": "ColorButton adds a colored button to the panel.\n\nIt is a
  "shell-version": [ "3.38", "40", "41.alpha" ],
  "url": "https://github.com/my-account/color-button",
  "session-modes": [ "unlock-dialog", "user" ]
}
```

Session Modes

In rare cases, it is necessary for an extension to continue running while the screen is locked. In order to be approved to use the unlock-dialog session mode:

- It **MUST** be necessary for the extension to operate correctly.
- All signals related to keyboard events **MUST** be disconnected in unlock-dialog session mode.
- The disable() function **MUST** have a comment explaining why you are using unlock-dialog .

Extensions **MUST NOT** disable selectively.

GSettings Schemas

For extensions that include a GSettings Schema:

- The Schema ID **MUST** use `org.gnome.shell.extensions` as a base ID.
- The Schema path **MUST** use `/org/gnome/shell/extensions` as a base path.
- The Schema XML file **MUST** be included in the extension ZIP file.
- The Schema XML filename **MUST** follow pattern of `<schema-id>.gschema.xml`.

Do not use telemetry tools

Extensions **MUST NOT** use any telemetry tools to track users and share the user data online.

Legal Restrictions

Code of Conduct

Extensions distributed on the extensions.gnome.org website are subject to the [Code of Conduct](#). While extensions may be used to download, access or operate on external content, anything distributed from GNOME infrastructure **MUST NOT** violate the CoC. This includes:

- Extension name and description
- Text content, icons, emojis and other media
- Screenshots

If an extension passes review, but contains content violating the Code of Conduct, please [open a Code of Conduct report](#).

Political Statements

Extensions **MUST NOT** promote national or international political agendas. A reviewer **MAY** request that particular content be removed before approval or decide to reject an extension entirely.

This policy exists to protect community members and users that live under duress of national and/or international sanctions. In some cases, even accessing parts of a website containing particular political sentiments is a criminal offense.

Licensing

GNOME Shell is licensed under the terms of the `GPL-2.0-or-later`, which means that derived works like extensions **MUST** be distributed under compatible terms (eg. `GPL-2.0-or-later`, `GPL-3.0-or-later`).

While your extension may include code licensed under a permissive license such as BSD/MIT, you are still approving GNOME to distribute it under terms compatible with the `GPL-2.0-or-later`.

If your extension contains code from another extension it **MUST** include attribution to the original author in the distributed files. Not doing so is a license violation and your extension will be rejected.

Copyrights and Trademarks

Extensions **MUST NOT** include copyrighted or trademarked content without proof of express permission from the owner. Examples include:

- Brand Names and Phrases
- Logos and Artwork
- Audio, Video or Multimedia

Recommendations

Don't include unnecessary files

Extension submissions should not include files that are not necessary for it to function. Examples include:

- build or install scripts

- .po and .pot files
- unused icons, images or other media

A reviewer **MAY** decide to reject an extension which includes an unreasonable amount of unnecessary data.

Use a linter

Using [ESLint](#) to check your code can catch syntax errors and other mistakes before submission, as well as enforce consistent code style. You can find the ESLint rules used by GNOME Shell [on GitLab](#).

Following a specific code style is not a requirement for approval, but if the codebase of an extension is too messy to properly review it **MAY** be rejected. This includes obfuscators and transpilers used with TypeScript.

UI Design

Although not required for approval, it is recommended that extension preferences follow the [GNOME Human Interface Guidelines](#) to improve consistency with the GNOME desktop.

Getting Help

There are several ways for you to get help with your extension.

- Ask in the extensions [Matrix](#) room
- Ask on [discourse.gnome.org](#) using the [extensions tag](#)
- Ask on StackOverflow using the [gnome-shell-extensions](#) and/or [gjs](#) tags

Previous page
[Legacy Documentation](#)

